
Testing Software Requirements

Part 1 - Importance

Importance of requirements

We have seen from earlier material/discussion that inadequate requirements are a commonly reported source of defects among experienced commercial developers.

How can this position be improved?

Poor quality requirements occur:

- partly because requirements elicitation and clarification is challenging;
- but also because some developers (perhaps unwittingly) choose to not invest sufficient time and effort into requirements.

How can requirements be improved?

Testing user stories?

Defining software requirements

- Broad spectrum of techniques for defining requirements.
- Range from very formal (i.e. mathematical notation) to very informal (sometimes only verbal)
- In most cases, requirements are necessarily complete.
- We are building something new
 - neither the client or the provider are entirely clear what is required
 - almost certainly, they will have a different perspective / objective.

Level of detail documented for software requirements: A spectrum

Title	User should be able to save story
Traditional User Story	As a user I want to save a story I'm reading Because I found it useful
Scenario	Given that I'm reading a story When I tap the <i>icon</i> to save a story Then save it to my 'Saved Stories'
Additional info	INFO <i>icon</i> = bookmark icon Design - [URL for design / assets]

User Stories
sometimes called
features,
sometimes called
requirements!

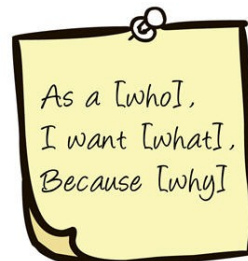
Non-verbalised
idea

Whiteboard
only

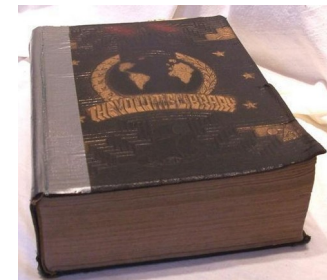
Detailed
User Stories

Verbalised
idea

Brief User
Stories –
post-it only

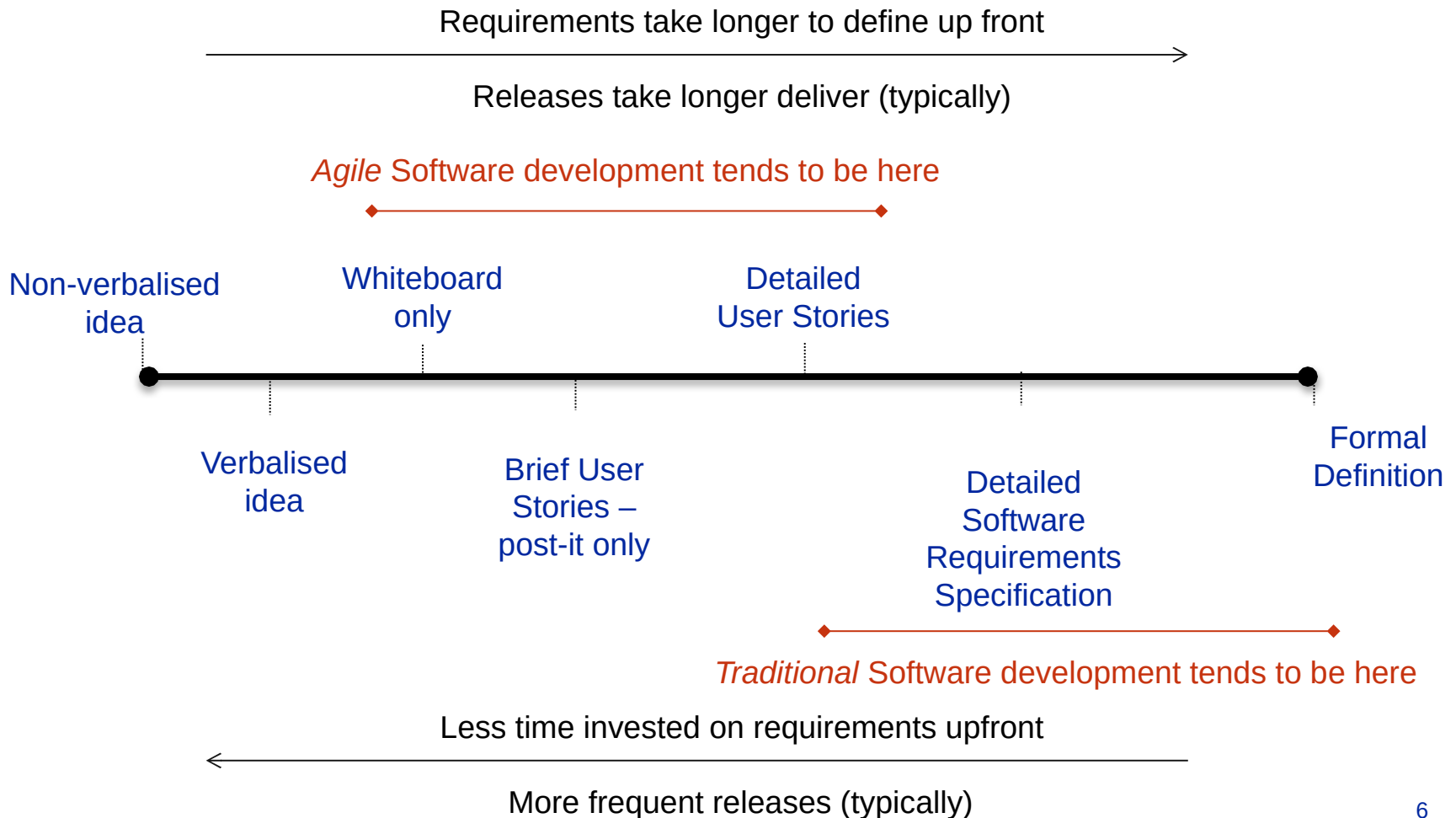


Detailed
Software
Requirements
Specification

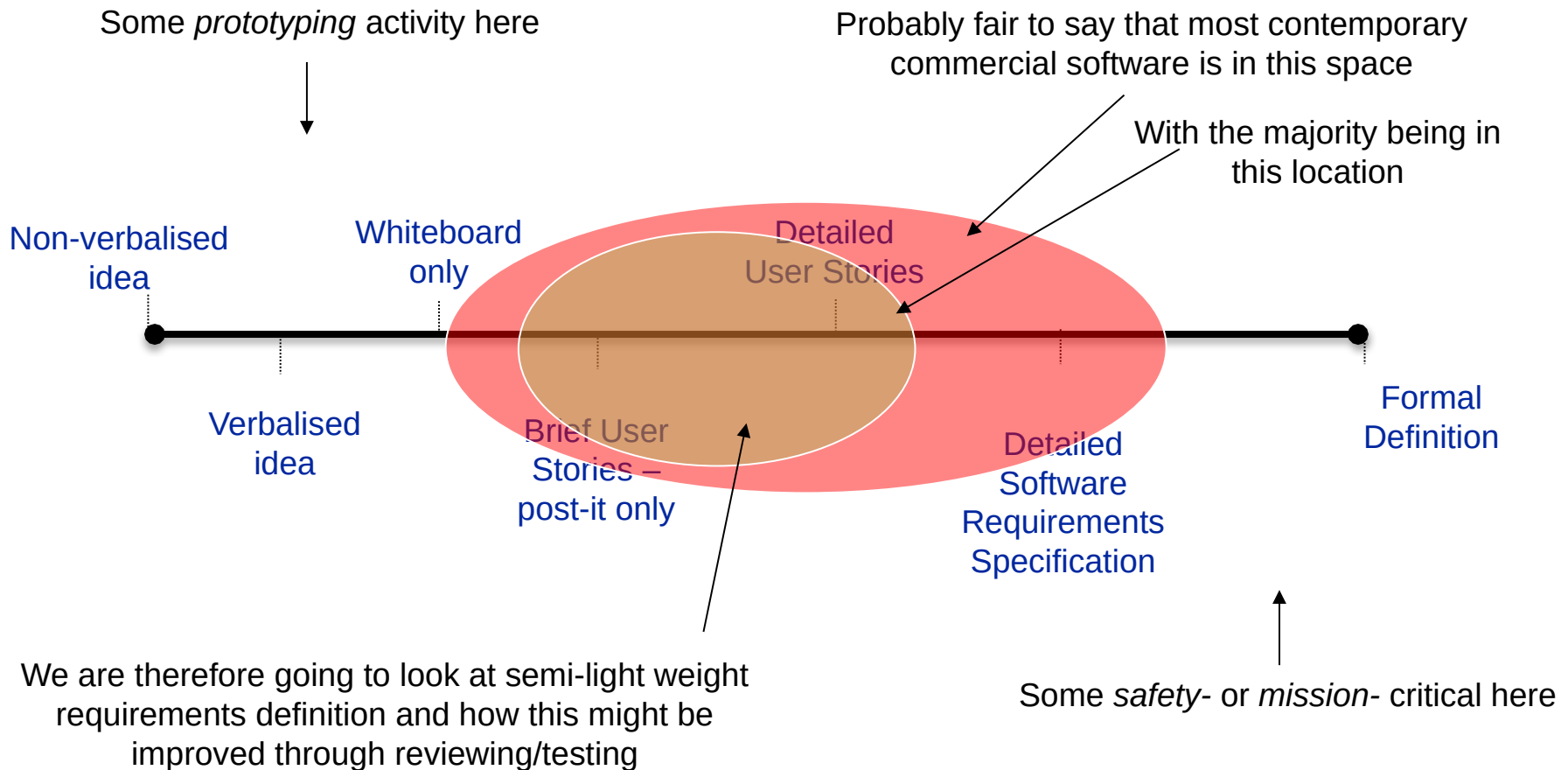


Formal
Definition

Interplay of software requirements with development methodology



Detail of software requirements: A spectrum



Part 2 – Problem

At the heart of the problem...

... we have human and other practical constraints.

We use natural language to express most things.

We assume that:

- We have a sufficiently complete understanding of some topic
- We have communicated our understanding completely to other concerned parties
- Others have completely understood our communication

There are many gaps in the natural language / human understanding / communication sphere.

At the heart of the problem...

- One of software's greatest strengths - its flexibility - is also the origin of some of its greatest challenges.
- Without the appropriate level of flexibility, software projects and companies can fail (and do fail).
- Some judgement is required in order to determine the *appropriate* level of requirement understanding and definition.
- This judgement is, to some extent, only learned through experience.
- But there are also good practices that we can adopt to assist us.

Those humans...

- Humans may be more inclined towards action rather than careful planning – there is not always a need in our day-to-day lives for heightened careful planning and retrospection.
- Hence, we may be optimised for operational type activities (i.e. problem -> fix; problem -> fix) rather than problem avoidance (i.e. more planning -> no problem / fewer problems).
- Most of the time for us humans, this operational mode is fine, in fact it might well be necessary.
- But for software development, failing to adequately work through requirements at the start, treating them as strategically important, can have devastating impacts for projects.

Strategic importance...

- Strategy Importance:
 - Worthy of investment
 - Worthy of careful planning and management
 - Critical to success
- Strategy V Tactics?

At the heart of the problem...

From a software development perspective, **we would benefit from being a little better at requirements quality.**

Evidence from a paper supports the case that **inadequately defined requirements may be the primary source of software defects** (at least that is what practitioners have reported).

On competition and winning business

Often starts from the point where things can be quite **nebulous**... Ideas, relationships, imperfect knowledge.

Can (and may have to?) win business on the basis of hazy s/w requirements.

New business development and sales-type activities are not overly concerned with engineering, delivery and lifetime maintenance of software/systems; rather they might be concerned with sales' commission, business survival, profitability and (in some cases) short term motivations prevail.

So we have **different concerns arising for different stakeholders** within software businesses.

Part 3 – Requirements

What are software requirements?

- In their infancy at least, they may be nothing more than **abstract concepts**.
- Converting these concepts into reliable working software is very challenging – there are many pitfalls and hurdles along the way.

Agile and non-agile worlds?

When compared with traditional approaches, agile software development breaks development down into much **smaller development cycles** and increases release frequency.

The agile approach is designed to **avoid lengthy phases of requirements documentation** and associated lengthy development cycles, by producing working software regularly thereby allowing users provide feedback on product development/direction.

This avoids the problem of investing large sums of money and long periods of time building products that do not satisfy clients / prospective users.

Agile and non-agile worlds?

Implemented correctly, agile software development can help to produce solutions that users actually want by building up small increments of functionality regularly and not going too far off course during implementation.

But agile is not a panacea for all requirements uncertainty.

In the wrong hands, agile software development can produce a lot of code that may not ultimately be required.

It becomes a matter of getting the balance right – between users story definition and timely feedback on working software.

Agile and non-agile worlds?

- Irrespective of the development philosophy – i.e. agile or traditional – each time **requirements** are defined and code is written, it **should be considered as an economic proposition**.
- Accordingly, we make an investment in requirements.
- Not all investments produce desirable levels of return!
- In all cases, we need to discover where we need to go as the project unfolds – **the destination does not exist at the outset, nor do all routes to getting there**.
- **Uncertainty prevails** – the only question relates to how we manage the uncertainty. This is a central challenge of all software development.

A simple requirement...

Requirement: *“I want to go from Paris to Cannes”*.

A simple requirement.

Or so it would seem....

An example with a simple
requirements.....

We all know where Cannes is...



'How' can this be achieved?



You can get the train.
You can drive.
You could walk!
You could fly.
Etc...

See anything (potentially) wrong so far?

- Is there an absence of detail in our requirement?
- Or do we have enough detail to start working through our solution?
- In what respect might this requirement be incomplete?

Let's revisit the requirement?

Requirement: *"I want to go from Paris to Cannes"*.

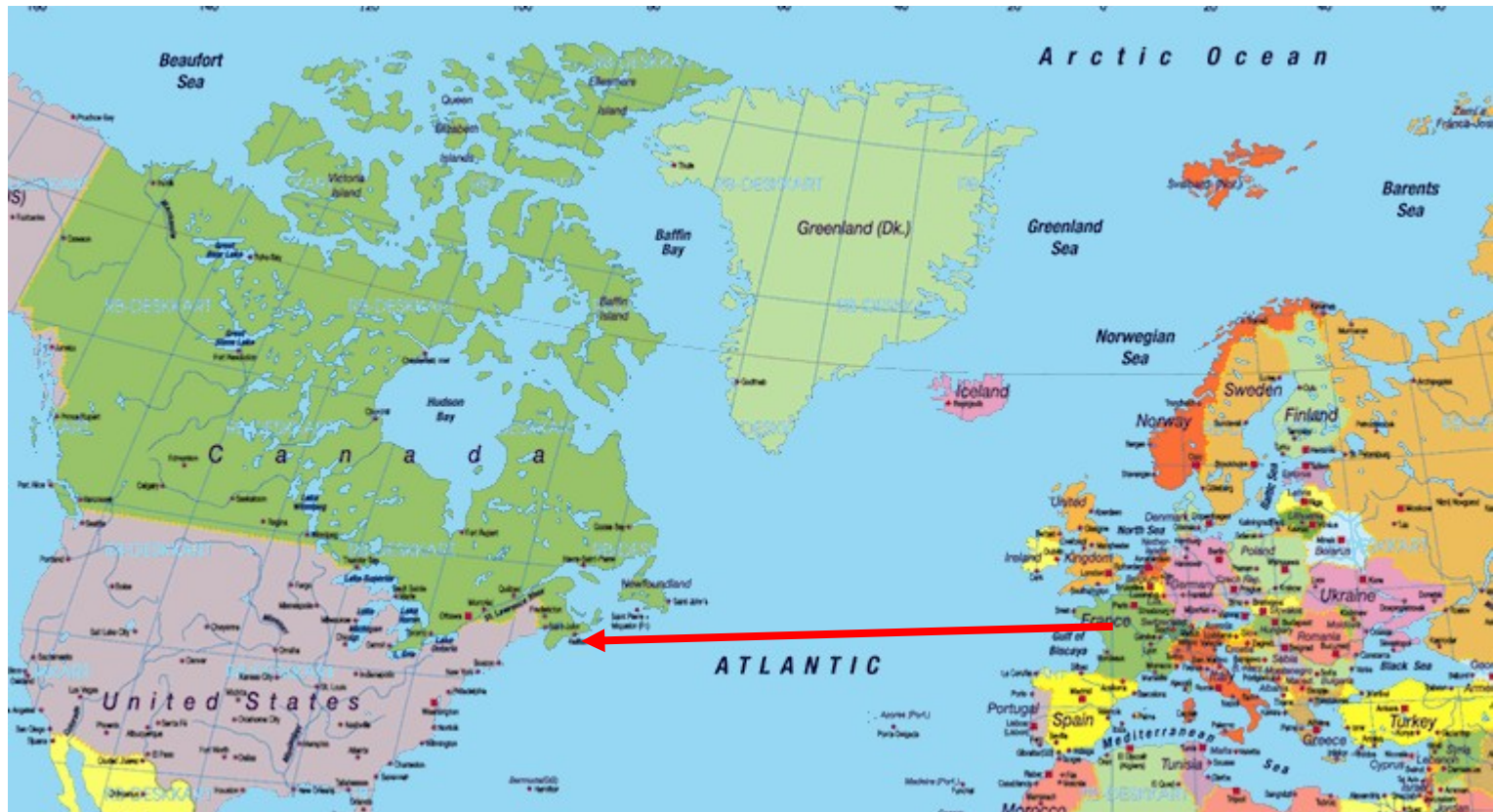
- Is it Cannes in Provence, France that you wish to travel to?
- Did I hear you correctly, was it Cannes, France or Caen, France?
- Did you mean Cannes, Nova Scotia, Canada?
- Or some other Cannes / Caen altogether?

Problem: We did not test our requirement for clarity/correctness before seeking out the solution to the assumed challenge.

Map of France revisited...



Partial map of the world...



Other potential requirement incompleteness...

- Did I mention that I want to visit my grandmother in Montpellier en route?
- Did I mention that I need to go to Cannes and return to Paris on the same day?
- Did you know that I have a fear of flying?
- Did you know that I only have €300 to spend on the trip?
- Did you know that I can only travel on Tuesdays?
- Did I tell you that I want my friend to accompany me?
- Did I note that my actual destination is actually 45kms west of Cannes?
- Etc, etc, etc.

There were so many things that we missed...

- Requirements should be **critically evaluated**.
- Requirements are critically evaluated in most successful software development companies.
- If we spend too much money building things that are not required, we are (i) at a competitive disadvantage, and (ii) risking project and company failure.
- The concern is not just a function of money, it is also a function of time.
- Requirements inherently have associated detail – the key question that arises is: **when** and **how** do we discover the necessary detail?

Relating the metaphor to software requirements?

Wrong place (i.e. Cannes / Caen)?

Basic functional defect

Need to visit my grandmother in Montpellier?

Functionally incomplete

Need to return the same day?

Performance requirement

Have a fear of flying?

Operational requirement?

Only have €300 to spend?

Project constraint

Can only go on a Tuesday?

Operational constraint?

Must be accompanied by friend?

Functionally incomplete

My actual destination is 45kms west of Cannes?

Functionally incomplete / inaccurate?

Is the seemingly simple requirement even viable?

Q. If I cannot fly,
and I need to return in the same day,
and I need to visit my grandmother in Montpellier en route,
and my meeting is actually 45kms west of Cannes,
and I want my friend to accompany me,
can I do all of this for €300? (i.e. considered collectively, there is some inconsistency emerging in our requirements)

- From what looked like a relatively simple requirement at the outset is starting to look like it might be impossible to achieve (or at the very least, it will be challenging to achieve).
- *Perhaps* it is theoretically possible in one day to drive if I own my own car: 7hrs 30mns Paris to Montpellier, visit grandmother for 1 hour, drive to my meeting, another 3hrs 30mns, meet for 1 hour, drive back to Paris, another 8hrs 15 mns.
- Simple things may become complex very quickly... some requirements are economically / practically unviable (but how/when do we discover this?).

Part 4 – More problems....

Exactly the same problems arise in software requirements

- It is difficult to evaluate software requirement quality.
- But there are some good frameworks that we can employ.

Testing Requirements

We will look at:

1. Quality of individual requirements / users stories
2. The quality of a group of software requirements / user stories when considered as a whole (i.e. as an overall product/application/system)

1. Quality of Individual Requirements

Evaluating Quality - Requirements

Is each quality requirement:

- **Mandatory:**
 - Not just an unintended architecture, design, or implementation constraint?
 - Relevant?
- **Feasible** given endeavour, technology, and physical limitations? Simple statements of goals such as “The system shall be secure” or “The system shall be reliable” are not good requirements because they are either infeasible because no system is totally secure or reliable or else ambiguous because they do not say how secure or reliable they need to be.
- **Scalable** so that it is clear just how much quality is required?
- **Unambiguous** so that all stakeholders and developers will interpret it the same way?
- **Verifiable** via testing, demonstration, inspection, etc?
- **Correct** in that it meets some real need of the stakeholders?
- **Prioritized** so that it can be allocated to an appropriate build or release?
- **Traced** to its source?

Does each quality requirement have an associated: **Rationale**?

Quality Software Requirements

- The previous source (i.e. paper from Journal Of Object Technology. Vol. 4, No. 9, November-December 2005) discusses the *quality* of quality requirements.
- How do these quality criteria manifest for the types of statements or user stories that are sometimes seen in agile software development?

Quality Requirement Statements (1)

Characteristics of Quality Requirement Statements:

Correct.

Each requirement must accurately describe the functionality to be delivered. The reference for correctness is the source of the requirement, such as an actual customer or a higher-level system requirements specification.

Only user representatives can determine the correctness of user requirements, which is why it is essential to include them, or their close surrogates, in inspections of the requirements. Requirements inspections that do not involve users can lead to developers saying, "That doesn't make sense. This is probably what they meant." This is also known as "**guessing**."

Quality Requirement Statements (2)

Characteristics of Quality Requirement Statements:

Feasible.

It must be possible to implement each requirement within the known capabilities and limitations of the system and its environment.

To avoid infeasible requirements, have a developer work with the requirements analysts or marketing personnel throughout the elicitation process.

This developer can **provide an early reality check on what can and cannot be done technically**, and what can be done only at excessive cost or with other tradeoffs.

Quality Requirement Statements (3)

Characteristics of Quality Requirement Statements:

Necessary.

Each requirement should document something the customers really need or something that is required for conformance to an external requirement, an external interface, or a standard.

Another way to think of "necessary" is that each requirement originated from a source you recognize as having the authority to specify requirements.

Trace each requirement back to its origin, such as a use case, system requirement, regulation, or some other voice-of-the-customer input.

If you cannot identify the origin, perhaps the requirement is an example of "gold plating" and is not really necessary.

Quality Requirement Statements (4)

Characteristics of Quality Requirement Statements:

Prioritised.

Assign an implementation priority to each requirement, feature, or use case to indicate how essential it is to include it in a particular product release.

Customers or their surrogates have the lion's share of the responsibility for establishing priorities.

If all the requirements are regarded as equally important, the project manager is less able to react to new requirements added during development, budget cuts, schedule overruns, or the departure of a team member.

Priority is a function of the value provided to the customer, the relative cost of implementation, and the relative technical risk associated with implementation.

Prioritisation levels can vary from setting to setting as appropriate to their needs.

Quality Requirement Statements (5)

Characteristics of Quality Requirement Statements:

Unambiguous.

The reader of a requirement statement should be able to **draw only one interpretation of it**. Also, multiple readers of a requirement should arrive at the same interpretation.

Natural language is highly prone to ambiguity. Avoid subjective words like user-friendly, easy, simple, rapid, efficient, several, state-of-the-art, improved, maximize, and minimize.

Write each requirement in succinct, simple, straightforward language of the user domain, not in *computerese*.

Effective ways to reveal ambiguity include formal inspections of the requirements specifications, writing test cases from requirements, and creating user scenarios that illustrate the expected behavior of a specific portion of the product.

Quality Requirement Statements (6)

Characteristics of Quality Requirement Statements:

Verifiable.

See whether you can devise tests or use other verification approaches, such as inspection or demonstration, to determine whether each requirement is properly implemented in the product.

If a requirement is not verifiable, determining whether it was correctly implemented is a matter of **opinion**.

Requirements that are not consistent, feasible, or unambiguous also are not verifiable.

Any requirement that says the product shall "support" something is not verifiable.

Quality Requirement Statements (7)

Testing the Quality of Requirement Statements:

“Another powerful quality technique is to write test cases against the requirements before you cut a single line of code. Test cases crystallize your vision of the product’s behavior as specified in the requirements and can reveal fuzziness, omissions, and ambiguities.”

This is part of Test Driven Development (TDD).

TDD is no less important in agile software development; rather it might be more critical for agile than for traditional software development?

Quality Requirement Statements (8)

Characteristics of Quality Requirement Statements:

Scalable.

Is each quality requirement:

- **Scalable** so that it is clear just how much quality is required?
- It might be possible to implement a single calculation or procedure in isolation, but can we perform 10,000 of these per second as might be required operationally
- Related to so-called *non-functional* perspectives: performance, security, safety
- Difficult to assess without experience, so we exclude this from our assessment.

Traceable.

Is the source of each quality requirement known?

- **Who** want this requirement / user story?
- If we do not know who wants it, then maybe it is not needed at all?
- We will deal with this at a system level – refer to later slides.

2. Quality of Collective Requirements

Requirements Quality - Collective

- **Overall** - Requirements statements are ultimately only a small part of an overall system / product.
- **Complete** - No requirements or necessary information should be missing.
- **Consistent**. Consistent **requirements do not conflict with other software requirements** or with higher level (system or business) requirements.
- **Modifiable**. You must be able to revise requirements when necessary and (potentially also) maintain a history of changes made to each requirement.
- **Traceable**. You should be able to link each software requirement to its **source**.

Requirements Quality - Overall (1)

- Requirements statements are ultimately only a small part of an overall system / product.
- We therefore need to also consider how individual requirement statements reside alongside other existing and new/changing requirements statements.
- Therefore we have to also look to **Characteristics of Quality Requirements Specifications (or our total set of statements / user stories)**

Requirements Quality - Overall (2)

Characteristics of Quality Requirements Specifications (or our total set of statements / user stories)

Complete. No requirements or necessary information should be missing. Completeness is also a desired characteristic of an individual requirement. It is hard to spot missing requirements because they aren't there.

If you focus on user tasks rather than on system functions during requirements elicitation, you are less likely both to overlook requirements and to include requirements that aren't really necessary.

If you know you are lacking certain information, use "TBD" ("to be determined") as a standard flag to highlight these gaps. Resolve all TBDs from a given set of the requirements before you proceed with construction of that part of the product.

Requirements Quality - Overall (3)

Characteristics of Quality Requirements Specifications (or our total set of statements / user stories)

Consistent. Consistent **requirements do not conflict with other software requirements** or with higher level (system or business) requirements.

Disagreements among requirements must be resolved before development can proceed. You may not know which (if any) is correct until you do some research.

Be careful when modifying the requirements, as inconsistencies can slip in undetected if you review only the specific change and not any related requirements.

Requirements Quality - Overall (4)

Characteristics of Quality Requirements Specifications (or our total set of statements / user stories)

Modifiable. You must be able to revise requirements when necessary and (potentially also) maintain a history of changes made to each requirement.

This may mean that each requirement be uniquely labelled and expressed separately from other requirements so you can refer to it unambiguously.

You can make groups of requirements more modifiable by organizing them so that **related requirements are grouped together**, and by creating a table of contents, index, and **cross-reference** listing.

Requirements Quality - Overall (5)

Characteristics of Quality Requirements Specifications (or our total set of statements / user stories)

Traceable. You should be able to link each software requirement to its **source**.

Also might need to link each software requirement to the design elements, source code, and test cases that are constructed to implement and verify the requirement (especially in safety critical software).

Traceable requirements are uniquely labelled and are written in a structured, fine-grained way, as opposed to large, narrative paragraphs or bullet lists.

Quality of Software Requirements

In summary we have: individual-statement and all-statements concerns

Individual Statements

Correct, Feasible, Necessary,
Prioritised, Unambiguous,
Verifiable

All Statements Together

Complete, Consistent,
Modifiable, Traceable,

Part 5 – Quality Examples

Examples

Let's take a look at some examples, all of which are based on Weigers' article.

Example 1

"The product shall provide status messages at regular intervals not less than every 60 seconds."

- This requirement is incomplete: what are the status messages and how are they supposed to be displayed to the user?
- The requirement contains several ambiguities. What part of "the product" are we talking about?
- Is the interval between status messages really supposed to be *at least* 60 seconds, so showing a new message every 10 years is okay?
- Perhaps the intent is to have no more than 60 seconds elapse between messages; would 1 millisecond be too short?
- The word "every" just confuses the issue. As a result of these problems, the requirement is not verifiable.

Example 1 (ctd)

Wieggers advocates splitting the requirement into multiple requirements because each will require separate test cases and because each should be separately traceable. If several requirements are strung together in a paragraph, it is easy to overlook one during construction or testing:

"1. Status Messages.

- 1.1. The Background Task Manager shall display status messages in a designated area of the user interface at intervals of 60 plus or minus 10 seconds.
- 1.2. If background task processing is progressing normally, the percentage of the background task processing that has been completed shall be displayed.
- 1.3. A message shall be displayed when the background task is completed.
- 1.4. An error message shall be displayed if the background task has stalled."

Example 1 (ctd)

There is much improvement in Wiegers' approach. But are some aspects still concerning?

"1. Status Messages.

1.1. The Background Task Manager ...

1.2. If background task processing is progressing **normally**, ...

1.3. A message shall be displayed when the background task is completed.

1.4. An error message shall be displayed if the background task has **stalled**."

This requirement rewrite clarifies the content and timing for status messages – which is welcome.

But what is *normal* and what do we mean by *stalled*?

In the case of 1.3, **for how long should the completed message be displayed**? Forever? For 60 plus or minus 10 seconds?

Therefore, room for significant improvement still exists in the quality of this requirement.

Agile considerations...

Some *Agile* interpretation of this scenario:

- We do not want to spend a very long time perfecting a requirement pre-implementation – we want to code and see what people think of our solution.
- The question becomes: **when is a requirement good enough?**
- And there is no simple answer to that question!
- At the very least, efforts should be taken to tease out the meaning/intent of individual requirements. And efforts to flesh out requirements are very much desirable as otherwise the fluid space between the requirement and the actual need can quickly become unmanageable.
- There is a question of technique also – if we **use TDD in combination with a unit testing strategy**, then we have a vehicle to challenge and improve certain aspects of requirements at a very early stage in implementation.
- In this view of the world, TDD with unit testing is partly requirements capture and validation.

Example 2

"Charge numbers should be validated on-line against the master corporate charge number list, if possible."

- This requirement is incomplete and unclear:
- What does "if possible" mean? If it's technically feasible? If the master charge number list can be accessed on line?
- Avoid imprecise words like "should." The customer either needs this functionality or she/he doesn't.
- "Shall" is a clearer statement of what is intended by the requirement and to explicitly specify priorities.
- Wieglers recommends: *"The system shall validate the charge number entered against the on-line master corporate charge number list. If the charge number is not found on the list, an error message shall be displayed and the order shall not be accepted."*
- Can/Should the quality of this revised requirement be further improved? How?
- Will the client lose potential business if the order "shall not be accepted"?

In review...

- We need to **be very careful with requirements**.
- Pay very careful attention to the **language** used.
- Ask: **can this requirement be tested?** (as a method to improving the quality of the requirement).
- Get used to **asking questions** such as “What do you mean by ...?”
- **Critically evaluate** the quality of each requirement, even if only informally.
- Pay particular attention to scantily defined requirements – **single, short sentences should carry ‘warning’ signs!** Most implementations are complex and therefore, we should not expect that can be adequately captured in single short sentences.
- **Use whiteboards and other tools** to share/refine thoughts...
- **Getting the balance right** in requirement elicitation and definition and quality prior to moving forward with implementation is one place where we can hope to improve
- It is not necessarily the case that everything has to be written down; in the spirit of agile, **it *might* be preferable/beneficial to discuss these matters for the purpose of clarification rather than an inefficient and time consuming textual exchange.**

Summary

When it comes to testing software requirements:

- We should **critically evaluate each and every requirement** (even if only briefly)
- The extent to which clarification & completeness are required is difficult to determine, so too is the level to which documentation is required.
- We should have **evaluation criteria**, things we think about when critically evaluating the quality of software requirements; e.g. complete, correct, testable, consistent, etc.
- There **are no shortcuts** to producing the software that is actually needed, and commencing with a foundation of poor quality requirements is likely to be one of worst approaches that can be adopted:
 - This can frustrate developers faced with constantly reworking old inadequate software
 - This does not please users / clients
 - This is not a sound economic proposition.

Critical analysis is difficult!

Think “Critical Analysis”. Q. What does this mean exactly?

- Think about the **intention/purpose** of the system and use this to guide your evaluation.
- Keep in mind the types of **constraints** that systems of this type are subject to.
- Don’t overlook **practicalities** and **inconsistencies**
- Ask: is **sufficient information** provided so as to test this requirement?
- Consider user stories **individually and collectively**
- Reviewing requirements is DIFFICULT, it is hard work, this is one of the reasons that it is such a major source of defects